In the previous section we configured data auditing for users and enforced a policy to return only the orders for the currently authenticated user. This section will show you how to test the data auditing as a slice test. To verify the data protection requirement, you can refer to the repository accompanying the book and check how it's been covered by the integration tests in the `OrderServiceApplicationTests` class (Chapter12/12-end/order-service/src/test/java).

Data auditing is applied at the repository level. We can extend the `OrderRepositoryR2dbcTests` class with extra test cases covering the scenario where the user is authenticated and when it is not.

Similar to what we did in Catalog Service, we can use the `@WithMockUser` annotation from Spring Security to execute a test method in an authenticated context, relying on a mock user representation.

Listing 12.33 Testing data auditing when users are authenticated or not

```
@DataR2dbcTest
@Import(DataConfig.class)
@Testcontainers
class OrderRepositoryR2dbcTests {
  ...

  @Test
  void whenCreateOrderNotAuthenticatedThenNoAuditMetadata() {
    var rejectedOrder = OrderService.buildRejectedOrder( "1234567890", 3);
    StepVerifier.create(orderRepository.save(rejectedOrder))
      .expectNextMatches(order -> Objects.isNull(order.createdBy()) &&
        Objects.isNull(order.lastModifiedBy()))                          ❶
      .verifyComplete();
  }

  @Test
  @WithMockUser("marlena")
  void whenCreateOrderAuthenticatedThenAuditMetadata() {
    var rejectedOrder = OrderService.buildRejectedOrder( "1234567890", 3);
    StepVerifier.create(orderRepository.save(rejectedOrder))
      .expectNextMatches(order -> order.createdBy().equals("marlena") &&
        order.lastModifiedBy().equals("marlena"))                        ❷
      .verifyComplete();
```